

ПРАВИТЕЛЬСТВО РОССИЙСКОЙ ФЕДЕРАЦИИ
ФГАОУ ВО НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
«ВЫСШАЯ ШКОЛА ЭКОНОМИКИ»

Факультет компьютерных наук
Образовательная программа «Прикладная математика и информатика»

Отчет о программном проекте на тему:
Разработка в условиях изменений требований

Выполнил студент:

группы #БПМИ2412, 2 курса

Круковер Денис Ильич

Принял руководитель проекта:

Кузнецов Михаил Александрович

Преподаватель

Факультет компьютерных наук НИУ ВШЭ

Содержание

Аннотация	5
1 Введение	5
2 Анализ предметной области и постановка задачи	6
2.1 Предметная область	6
2.2 Проблема и актуальность	6
2.3 Постановка задачи	7
3 Техническое задание	8
1. Назначение продукта	8
1.1. Термины и определения	8
2. Сценарии использования	9
2.1. Основной пользовательский сценарий	9
2.2. Сценарий автора	9
2.3. Сценарии администратора	9
3. Границы проекта	10
3.1. Входит в проект	10
3.2. Не входит в проект	10
4. Роли и правила доступа	10
4.1. Роли	10
4.2. Общие правила доступа	11
5. Общие требования к данным	11
5.1. Общие правила валидации	11
5.2. Общие правила для текстовых полей	11
5.2.1. Термины удаления	12
5.3. Связи сущностей	12
6. Функциональные требования по доменам	12
6.1. Пользователи и профиль	12
6.2. Маршруты	16
6.3. Точки маршрута	23
6.4. Впечатления и модерация	27
6.5. Витрина впечатлений	34

6.6. Покупки	35
6.7. Отзывы	38
6.8. Рекомендации	40
6.9. Прогресс и realtime	42
6.10. Аналитика	45
6.11. BFF	48
6.12. Интеграции	49
7. Общие требования к API	50
7.1. Формат ответов	50
7.1.1. Основные коды ошибок	51
7.2. Требования к API	52
7.2.1. Общие правила пагинации и сортировки	52
7.3. Список требуемых API	52
8. Нефункциональные требования	54
8.1. Требования к отклику	54
8.2. Общие требования к качеству	55
9. Требования к проверке качества	55
9.1. Общий подход	55
9.2. Минимальный состав проверок для методики испытаний	55
9.3. Сценарии, подлежащие отражению в методике испытаний	55
9.4. Критерии успешности проверки качества	57
4 Описание решения	57
4.1 Общий результат	57
4.2 Выбор технологий	57
4.3 Архитектура приложения	58
4.4 Модель данных	59
4.5 Реализованные пользовательские сценарии	60
4.6 API-контракт и обработка ошибок	61
4.7 Realtime-прогресс	61
4.8 Рекомендации, витрина и аналитика	62
4.9 Проверка качества	62
5 Рефлексия проделанной работы	67

6	Заключение	67
7	Использование инструментов искусственного интеллекта	68
A	Структура реализованных API-доменов	69

Аннотация

В работе описана разработка backend-сервиса для трэвел-приложения, которое позволяет пользователям создавать городские маршруты, публиковать на их основе готовые впечатления, покупать их и проходить маршрут с фиксацией прогресса. Основным результатом проекта - работающий прототип серверной части, реализованный на FastAPI с асинхронным доступом к базе данных через SQLAlchemy и PostgreSQL. Система включает регистрацию и авторизацию пользователей, разделение ролей, управление маршрутами и точками маршрута, модерацию, витрину впечатлений, покупки через рау-заглушку, отзывы, рекомендации, realtime-обновление прогресса, аналитику и BFF-методы для будущего frontend. Отдельное внимание уделено формальному API-контракту: все ответы возвращаются в едином формате, ошибки нормализованы, а основные пользовательские сценарии проверены автоматическими тестами.

1 Введение

Современные городские сервисы всё чаще ориентированы не только на поиск отдельных мест, но и на создание цельного пользовательского опыта: прогулки, экскурсии, тематического маршрута или самостоятельного путешествия. Для такого продукта недостаточно хранить список достопримечательностей. Требуется backend, который связывает маршруты, авторов, покупки, прохождение, отзывы, рекомендации и аналитику в единую систему.

Целью курсового проекта является разработка серверной части трэвел-приложения, в котором пользователь может создавать маршруты, превращать их в готовые впечатления, публиковать их в витрине, покупать опубликованные впечатления и проходить их с сопровождением по точкам маршрута. Задача является программным проектом: основным результатом работы - не исследование алгоритма, а реализованный прототип системы с формализованным API, моделью данных, правилами доступа и проверками качества.

В ходе работы были сформулированы требования к продукту, выделены основные роли пользователей и домены системы, спроектирована структура REST API, реализованы серверные модули и автоматические проверки ключевых сценариев. Разработка велась по техническому заданию проекта. При реализации использованы FastAPI, SQLAlchemy, PostgreSQL и JWT-механизм авторизации.

2 Анализ предметной области и постановка задачи

2.1 Предметная область

Предметная область проекта - цифровые сервисы для городских прогулок и путешествий. Пользователь такого сервиса ожидает, что приложение не просто покажет набор мест на карте, а предложит законченное впечатление: маршрут с последовательностью точек, описанием, длительностью, ценой, отзывами и понятным процессом прохождения.

В рамках проекта были выделены следующие базовые сущности:

- маршрут - последовательность точек, которую пользователь проходит в определённом порядке
- точка маршрута - географическая точка с координатами, описанием и порядковым номером
- впечатление - опубликованный пользовательский продукт, созданный на основе маршрута
- витрина - публичный каталог опубликованных впечатлений
- покупка - запись о приобретении впечатления пользователем
- прогресс - состояние прохождения впечатления пользователем
- аналитическое событие - запись о значимом действии пользователя или системы

Ключевая особенность предметной области состоит в том, что одна и та же сущность должна поддерживать разные состояния жизненного цикла. Например, маршрут может быть черновиком, находиться на модерации, быть опубликованным или архивированным. Поэтому backend должен не только выполнять CRUD-операции, но и контролировать допустимые переходы статусов, права доступа и связанность данных.

2.2 Проблема и актуальность

Обычный каталог мест плохо покрывает сценарий самостоятельной прогулки: пользователь вынужден сам выбирать порядок посещения точек, сверять расстояния, искать описания и понимать, завершил ли он маршрут. С другой стороны, авторам городских прогулок нужен инструмент публикации готовых впечатлений, а администраторам - механизм контроля качества и безопасности контента.

Практическая ценность проекта заключается в создании backend-основы, на которую можно подключить мобильный или web-клиент. Такая серверная часть берёт на себя хранение данных, правила доступа, публикацию контента, оплату через заглушку, realtime-прогресс и сбор аналитики. Это снижает сложность будущего frontend и позволяет развивать продукт по доменам.

2.3 Постановка задачи

Необходимо разработать backend-сервис, который предоставляет API для следующих групп пользователей:

- обычный пользователь регистрируется, просматривает витрину, покупает и проходит впечатления, оставляет отзывы
- автор создаёт маршруты и впечатления, а после верификации публикует их без предварительной модерации
- администратор модерирует маршруты и впечатления, блокирует пользователей, назначает роль автора и просматривает аналитику

Сервис должен обеспечивать единый формат ответов, проверку входных данных, авторизацию по JWT, хранение данных в реляционной базе, realtime-обновление прогресса через WebSocket и набор автоматических тестов для основных сценариев.

3 Техническое задание

1. Назначение продукта

Разрабатывается backend-сервис для трэвел-приложения, которое помогает пользователю получать новые впечатления во время поездок, прогулок и путешествий по городу.

1.1. Термины и определения

В рамках данного ТЗ используются следующие определения:

маршрут	последовательность точек, которые предлагаются пользователю пройти в определённом порядке
точка маршрута	отдельная географическая точка внутри маршрута с координатами, описанием и порядком прохождения
впечатление	готовый пользовательский продукт, построенный на основе маршрута и подготовленный для показа в витрине, покупки и прохождения
витрина	каталог опубликованных впечатлений, доступных для просмотра, фильтрации, выбора и покупки
верифицированный пользователь	пользователь, прошедший проверку и получивший роль <code>author</code>
профиль пользователя	набор пользовательских данных, который хранится в системе и возвращается после авторизации
покупка	запись о том, что пользователь инициировал или успешно завершил приобретение впечатления
прогресс	текущее состояние прохождения впечатления пользователем
рекомендации	список впечатлений, подобранных системой по правилам, указанным в разделе 6.8
аналитическое событие	запись о действиях пользователя или системы, события указаны в разделе 6.10
BFF	набор API-методов, которые собирают данные из нескольких частей backend и возвращают их в форме, удобной для frontend

Продукт должен решать следующие задачи:

1. пользователь должен иметь возможность создавать маршруты
2. пользователь должен иметь возможность превращать маршрут в готовое впечатление

3. пользователь должен иметь возможность просматривать витрину опубликованных впечатлений
4. пользователь должен иметь возможность покупать впечатления
5. пользователь должен иметь возможность проходить впечатление с сопровождением по маршруту
6. система должна формировать рекомендации
7. система должна фиксировать основные пользовательские действия

2. Сценарии использования

2.1. Основной пользовательский сценарий

1. пользователь регистрируется и входит в систему
2. пользователь открывает витрину впечатлений
3. пользователь просматривает карточку впечатления
4. пользователь покупает впечатление (впечатление необязательно платное, оно может быть и бесплатным)
5. пользователь запускает прохождение впечатления
6. клиент передаёт координаты пользователя
7. система определяет прогресс прохождения маршрута
8. пользователь завершает прохождение
9. в течение прохождения этапов система фиксирует события аналитики

2.2. Сценарий автора

1. автор создаёт маршрут
2. автор добавляет точки маршрута
3. автор отправляет маршрут на публикацию
4. маршрут автора публикуется сразу после отправки
5. автор создаёт впечатление на основе маршрута
6. автор отправляет впечатление на публикацию
7. впечатление автора публикуется сразу после отправки
8. впечатление автора сразу появляется в витрине

2.3. Сценарии администратора

1. администратор просматривает маршруты и впечатления пользователей в статусе `on_moderation`

2. администратор публикует или отклоняет маршруты и впечатления неverified пользователей
3. администратор блокирует или разблокирует пользователя
4. администратор переводит пользователя в статус verified автора
5. администратор имеет доступ к аналитическим данным

3. Границы проекта

3.1. Входит в проект

1. регистрация и авторизация пользователей
2. разделение прав ролей `user`, `author`, `admin` (права на каждый API-метод описаны в 7.3)
3. управление маршрутами
4. управление точками маршрута
5. создание и редактирование впечатлений
6. модерация маршрутов и впечатлений
7. витрина впечатлений
8. покупка впечатлений через pay-заглушку
9. отзывы и оценки
10. рекомендации без ML
11. отслеживание прогресса маршрута в реальном времени
12. аналитика действий пользователя
13. BFF API-методы для будущего frontend
14. проксирование внешних API, таких как картографических

3.2. Не входит в проект

1. реальные платёжные системы
2. офлайн-навигация
3. ML-рекомендации
4. социальная сеть
5. сложная мультимедийная платформа

4. Роли и правила доступа

4.1. Роли

1. **user**: просматривает витрину, покупает и проходит впечатления, оставляет отзывы, создаёт маршруты и впечатления, которые публикуются только после модерации
2. **author**: является верифицированным пользователем и имеет все возможности **user**, но его маршруты и впечатления публикуются сразу без предварительной модерации
3. **admin**: имеет все возможности **author**, а также выполняет модерацию, блокировку пользователей и просмотр аналитики

4.2. Общие правила доступа

1. публичные API-методы не требуют авторизации
2. защищённые API-методы требуют валидного JWT (неистёкшего по времени)
3. изменение сущности разрешено только владельцу сущности либо **admin**
4. административные API-методы доступны только **admin**
5. пользователь не должен получать доступ к чужим покупкам, прогрессу и черновикам
6. в витрине показываются только впечатления в статусе **published**
7. маршруты и впечатления пользователя с ролью **user** должны проходить модерацию перед публикацией
8. маршруты и впечатления пользователя с ролью **author** публикуются сразу
9. WebSocket-подключение должно быть доступно только авторизованному пользователю
10. заблокированный пользователь не должен иметь доступ к защищённым API-методам даже при наличии валидного JWT

5. Общие требования к данным

5.1. Общие правила валидации

1. все входные данные должны проходить серверную валидацию до выполнения бизнес-логики (пункты 5.1 и 7.2)
2. все строковые поля должны обрезаться по краям от лишних пробелов перед сохранением
3. пустые строки не должны приниматься как валидные значения обязательных полей
4. идентификаторы должны передаваться как положительные целые числа
5. даты и время должны передаваться и возвращаться в ISO 8601 (YYYY-MM-DDThh:mm:ss)
6. ошибки валидации должны возвращаться в предсказуемом формате (пункт 7.1)

5.2. Общие правила для текстовых полей

1. названия маршрутов, впечатлений и точек должны содержать от 3 до 120 символов
2. описания маршрутов и впечатлений должны содержать от 10 до 2000 символов

3. комментарий в отзыве должен содержать от 1 до 1000 символов
4. текстовые поля должны допускать буквы кириллицы и латиницы, цифры, пробел, а также базовые знаки пунктуации
5. управляющие символы и пустые значения должны отклоняться

5.2.1. Термины удаления

В рамках ТЗ используются следующие трактовки удаления:

1. **удаление:** пользовательский сценарий удаления сущности из интерфейса или API
2. **физическое удаление:** техническая операция, при которой запись удаляется из базы данных без сохранения
3. **архивирование:** мягкое удаление, при котором запись сохраняется в базе данных и переводится в статус `archived`

5.3. Связи сущностей

В системе фиксируются следующие основные связи между сущностями:

1. `User1:NRoute`
2. `Route1:NRoutePoint`
3. `Route1:NImpression`
4. `User1:NPurchase`
5. `Impression1:NReview`
6. `User1:NReview`
7. `User1:NAnalyticsEvent`
8. `User1:NProgress`

6. Функциональные требования по доменам

В рамках данного ТЗ доменом называется функциональная область системы, объединяющая близкие пользовательские сценарии, сущности и связанные с ними API-методы. Например, сценарии создания маршрута, редактирования маршрута и отправки маршрута на публикацию относятся к домену маршрутов.

6.1. Пользователи и профиль

6.1.1. Определение профиля

Профиль пользователя в рамках системы представляет собой совокупность пользовательских данных, которые backend хранит и возвращает после авторизации. Профиль не создаётся отдельным API-методом после регистрации, а возникает как результат регистрации пользователя.

6.1.2. Состав данных пользователя

Пользователь должен содержать следующие поля:

1. `id`
2. `username`
3. `email`
4. `password_hash`
5. `role`
6. `is_blocked`
7. `created_at`
8. `updated_at`

6.1.3. Пример JSON профиля

```
{
  "id": 1,
  "username": "for_example123",
  "email": "user@example.com",
  "role": "user",
  "is_blocked": false,
  "created_at": "2026-04-08T10:00:00Z",
  "updated_at": "2026-04-08T10:00:00Z"
}
```

6.1.4. Правила валидации

1. `username` обязателен
2. `username` должен быть уникальным
3. `username` должен содержать от 3 до 30 символов
4. `username` должен содержать только латинские буквы, цифры и символ подчёркивания `_`
5. `email` обязателен и должен соответствовать формату email (базовая валидация email)
6. `email` должен быть уникальным
7. `password` обязателен
8. длина `password` должна быть от 8 до 64 символов
9. `password` должен содержать хотя бы одну букву, хотя бы одну цифру и хотя бы один специальный символ
10. `role` должен принадлежать одному из значений: `user`, `author`, `admin`

11. при публичной регистрации пользователю назначается роль **user**
12. роль **author** назначается только после верификации администратором
13. пароль должен храниться только в виде безопасного криптографического хэша

6.1.5. Функциональные требования

1. система должна обеспечивать регистрацию пользователя
2. система должна обеспечивать авторизацию пользователя
3. система должна обеспечивать получение профиля текущего пользователя
4. система должна обеспечивать блокировку и разблокировку пользователя администратором
5. система должна обеспечивать перевод пользователя в роль **author** после верификации администратором

6.1.6. API-методы домена

```
POST /auth/register
```

Вход:

1. **username**
2. **email**
3. **password**

Что происходит:

1. проверяется уникальность **username**
2. проверяется уникальность **email**
3. валидируется пароль
4. пароль хэшируется
5. создаётся пользователь
6. пользователю назначается роль **user**
7. возвращается созданный профиль пользователя

Выход:

1. **access_token**
2. тип токена
3. идентификатор пользователя
4. **username**
5. **email**

6. роль

POST /auth/login

Вход:

1. email или username
2. password

Что происходит:

1. система проверяет существование пользователя
2. система сравнивает пароль с хэшем
3. при успехе выдаётся JWT

Выход:

1. access_token
2. тип токена
3. идентификатор пользователя
4. username
5. email
6. роль

GET /users/me

Вход:

1. JWT текущего пользователя

Что происходит:

1. система определяет пользователя по токену
2. возвращает профиль текущего пользователя

Выход:

1. JSON профиля пользователя (пункт 6.1.3)

PATCH /admin/users/{user_id}/block

Вход:

1. идентификатор пользователя

2. флаг блокировки
3. JWT администратора

Что происходит:

1. проверяется роль `admin`
2. обновляется признак блокировки пользователя

Выход:

1. обновлённый профиль пользователя

```
PATCH /admin/users/{user_id}/verify-author
```

Вход:

1. идентификатор пользователя
2. JWT администратора

Что происходит:

1. проверяется роль `admin`
2. пользователю назначается роль `author`

Выход:

1. обновлённый профиль пользователя

6.2. Маршруты

6.2.1. Назначение

Маршрут представляет собой последовательность точек, которые пользователь должен пройти в определённом порядке. Маршрут является основой для создания впечатления.

6.2.2. Состав данных маршрута

Маршрут должен содержать следующие поля:

1. `id`
2. `user_id`
3. `name`
4. `description`
5. `cities`
6. `duration_minutes`

7. status
8. moderation_comment
9. created_at
10. updated_at

6.2.3. Источник данных для городов

Поле `cities` должно заполняться пользователем при создании или редактировании маршрута.

Для работы с городами действуют следующие правила:

1. пользователь выбирает города из справочника одного выбранного картографического провайдера
2. backend использует `external_city_id` этого провайдера как основной идентификатор города
3. для каждого города backend сохраняет как минимум `external_city_id` и `name`
4. backend проверяет, что переданный город действительно существует в используемом справочнике провайдера
5. для одного маршрута не должны сохраняться дублирующиеся города с одинаковым `external_city_id`

6.2.4. Пример JSON маршрута

```
{
  "id": 10,
  "user_id": 3,
  "name": "Прогулка по центру Москвы",
  "description": "Маршрут на два часа по основным точкам центра",
  "cities": [
    {
      "external_city_id": "12345",
      "name": "Москва"
    }
  ],
  "duration_minutes": 120,
  "status": "published",
  "moderation_comment": null,
  "created_at": "2026-04-08T10:00:00Z",
  "updated_at": "2026-04-08T10:00:00Z"
}
```

6.2.5. Статусы маршрута

Маршрут должен поддерживать следующие статусы:

1. `draft`
2. `on_moderation`
3. `published`
4. `rejected`
5. `archived`

Статус `draft` означает, что пользователь ещё редактирует маршрут и не отправил его на публикацию. Пока маршрут находится в `draft`, он не попадает на модерацию и не может быть показан в публичных списках.

6.2.5.1. Разрешённые переходы статуса

Для маршрута должны поддерживаться следующие переходы:

1. `draft`->`on_moderation` после `submit`, если владелец имеет роль `user`
2. `draft`->`published` после `submit`, если владелец имеет роль `author` или `admin`
3. `on_moderation`->`published` по решению `admin`
4. `on_moderation`->`rejected` по решению `admin`
5. `rejected`->`draft` после доработки владельцем
6. `rejected`->`on_moderation` или `rejected`->`published` при повторном `submit` без доработки, в зависимости от роли владельца
7. `published`->`archived` по решению владельца или `admin`

При редактировании маршрута в статусе `published` маршрут должен переводиться обратно в `draft`.

6.2.6. Правила валидации

1. `name` обязателен
2. `name` должен содержать от 3 до 120 символов
3. `description` обязателен
4. `description` должен содержать от 10 до 2000 символов
5. `cities` должен быть непустым массивом объектов города
6. в массиве `cities` должно быть от 1 до 10 значений
7. каждый объект города должен содержать `external_city_id` и `name`
8. `external_city_id` должен быть непустой строкой
9. `name` должен содержать от 2 до 80 символов

10. значения в `cities` должны храниться без дубликатов по `external_city_id`
11. отображаемое поле `name` должно нормализоваться по пробелам
12. `duration_minutes` должно быть целым числом от 1 до 1440
13. `status` должен принадлежать фиксированному набору значений
14. `moderation_comment` может быть пустым, но при отклонении маршрута должен содержать от 1 до 1000 символов

6.2.7. Функциональные требования

1. система должна обеспечивать создание маршрута
2. система должна обеспечивать сохранение маршрута в статусе `draft`
3. система должна обеспечивать получение списка маршрутов
4. система должна обеспечивать получение маршрута по идентификатору
5. система должна обеспечивать редактирование маршрута
6. система должна обеспечивать удаление маршрута
7. система должна обеспечивать отправку маршрута на публикацию
8. система должна обеспечивать перевод маршрута пользователя в статус `on_moderation` только после отправки маршрута на публикацию
9. система должна обеспечивать публикацию маршрута автора после отправки маршрута на публикацию
10. система должна обеспечивать публикацию и отклонение маршрута администратором
11. система должна обеспечивать возврат отклонённого маршрута в `draft` после доработки
12. система должна обеспечивать перевод опубликованного маршрута в `draft` при его редактировании
13. система должна обеспечивать архивирование маршрута вместо физического удаления, если маршрут уже был опубликован или связан с впечатлениями
14. система должна запрещать редактирование маршрута, если с ним связано хотя бы одно впечатление в статусе `published`
15. система должна запрещать удаление и архивирование маршрута, если с ним связано хотя бы одно впечатление в статусе `published`
16. система должна возвращать понятную ошибку при отказе в редактировании, удалении или архивировании маршрута с указанием причины отказа, если с маршрутом связано опубликованное впечатление

6.2.8. API-методы домена

POST /routes

Вход:

1. `name`
2. `description`
3. `cities`
4. `duration_minutes`
5. JWT авторизованного пользователя

Что происходит:

1. валидируются поля маршрута
2. маршрут связывается с текущим пользователем
3. маршрут создаётся в статусе **draft**
4. маршрут не отправляется на модерацию автоматически
5. маршрут сохраняется в базе

Выход:

1. JSON созданного маршрута

POST /routes/{route_id}/submit

Вход:

1. идентификатор маршрута
2. JWT владельца маршрута

Что происходит:

1. проверяется, что маршрут принадлежит текущему пользователю
2. проверяется, что маршрут находится в статусе **draft** или **rejected**
3. проверяется, что маршрут содержит хотя бы одну точку
4. если владелец маршрута имеет роль **user**, статус меняется на **on_moderation**
5. если владелец маршрута имеет роль **author** или **admin**, статус меняется на **published**

Выход:

1. обновлённый JSON маршрута

GET /routes

Вход:

1. параметры пагинации
2. фильтр по `external_city_id`
3. фильтр по автору

Что происходит:

1. в публичном режиме система возвращает только маршруты со статусом `published`
2. владелец маршрута может получить свои маршруты в любых собственных статусах
3. администратор может получить маршруты всех статусов

Выход:

1. массив маршрутов
2. служебные поля пагинации

GET /routes/{route_id}

Вход:

1. идентификатор маршрута

Что происходит:

1. система получает маршрут
2. система проверяет право доступа к текущему статусу маршрута
3. система возвращает маршрут вместе со списком его точек

Выход:

1. JSON маршрута
2. список точек маршрута

PATCH /routes/{route_id}

Вход:

1. обновляемые поля маршрута
2. JWT владельца маршрута или `admin`

Что происходит:

1. проверяется право на изменение
2. валидируются новые данные
3. проверяется, что с маршрутом не связано ни одного впечатления в статусе **published**
4. если маршрут находится в статусе **published**, он переводится в **draft**
5. если маршрут находится в статусе **rejected**, он остаётся в **draft** после сохранения изменений
6. маршрут обновляется

Если с маршрутом связано хотя бы одно впечатление в статусе **published**, система должна вернуть понятную ошибку с указанием причины отказа.

Выход:

1. обновлённый JSON маршрута

```
DELETE /routes/{route_id}
```

Вход:

1. идентификатор маршрута
2. JWT владельца маршрута или **admin**

Что происходит:

1. проверяется право на удаление
2. проверяется, что с маршрутом не связано ни одного впечатления в статусе **published**
3. если маршрут находится в статусе **draft** и не связан ни с одним впечатлением, он может быть удалён физически
4. во всех остальных случаях маршрут переводится в статус **archived**

Если с маршрутом связано хотя бы одно впечатление в статусе **published**, система должна вернуть понятную ошибку с указанием причины отказа.

Выход:

1. служебный результат об успешном удалении

```
GET /moderation/routes
```

Вход:

1. JWT администратора

Что происходит:

1. система возвращает маршруты в статусе **on_moderation**

Выход:

1. массив маршрутов в статусе `on_moderation`

```
POST /moderation/routes/{route_id}/publish
```

Вход:

1. идентификатор маршрута
2. JWT администратора

Что происходит:

1. проверяется роль `admin`
2. проверяется, что маршрут находится в статусе `on_moderation`
3. проверяется, что маршрут содержит хотя бы одну точку
4. статус маршрута меняется на `published`

Выход:

1. обновлённый JSON маршрута

```
POST /moderation/routes/{route_id}/reject
```

Вход:

1. идентификатор маршрута
2. причина отклонения
3. JWT администратора

Что происходит:

1. проверяется роль `admin`
2. проверяется, что маршрут находится в статусе `on_moderation`
3. статус маршрута меняется на `rejected`
4. причина отклонения сохраняется в `moderation_comment`

Выход:

1. обновлённый JSON маршрута

6.3. Точки маршрута

6.3.1. Назначение

Точка маршрута описывает конкретную географическую точку, которую пользователь должен посетить при прохождении маршрута.

6.3.2. Состав данных точки

Точка маршрута должна содержать следующие поля:

1. id
2. route_id
3. name
4. latitude
5. longitude
6. description
7. point_order
8. stay_minutes

6.3.3. Пример JSON точки маршрута

```
{
  "id": 101,
  "route_id": 10,
  "name": "Красная площадь",
  "latitude": 55.7539,
  "longitude": 37.6208,
  "description": "Стартовая точка маршрута",
  "point_order": 1,
  "stay_minutes": 20
}
```

6.3.4. Правила валидации

1. name обязателен
2. name должен содержать от 3 до 120 символов
3. latitude должна быть числом в диапазоне от -90 до 90
4. longitude должна быть числом в диапазоне от -180 до 180
5. description должен содержать от 0 до 1000 символов
6. point_order должен быть целым числом от 1 и выше
7. внутри одного маршрута не может быть двух точек с одинаковым point_order
8. stay_minutes должно быть целым числом от 0 до 1440
9. в одном маршруте должно быть не более 100 точек

6.3.5. Функциональные требования

1. система должна обеспечивать добавление точки в маршрут
2. система должна обеспечивать редактирование точки маршрута
3. система должна обеспечивать удаление точки маршрута
4. система должна обеспечивать получение списка точек маршрута
5. система должна обеспечивать сохранение порядка точек
6. система должна запрещать изменение точек маршрута, если редактирование самого маршрута запрещено из-за связанного впечатления в статусе **published**
7. система должна ограничивать количество точек одного маршрута значением 100 для MVP

6.3.6. API-методы домена

```
POST /routes/{route_id}/points
```

Вход:

1. **name**
2. **latitude**
3. **longitude**
4. **description**
5. **point_order**
6. **stay_minutes**
7. JWT владельца маршрута или **admin**

Что происходит:

1. проверяется существование маршрута
2. проверяется право на изменение маршрута
3. проверяется, что редактирование маршрута не запрещено бизнес-правилами
4. проверяется, что количество точек маршрута не превышает установленный лимит
5. валидируются координаты и порядок
6. точка добавляется в маршрут

Выход:

1. JSON созданной точки

```
GET /routes/{route_id}/points
```

Вход:

1. идентификатор маршрута

Что происходит:

1. система возвращает точки маршрута в порядке прохождения

Выход:

1. массив точек маршрута

```
PATCH /routes/{route_id}/points/{point_id}
```

Вход:

1. изменяемые поля точки
2. JWT владельца маршрута или `admin`

Что происходит:

1. проверяется право доступа
2. проверяется, что редактирование маршрута не запрещено бизнес-правилами
3. валидируются данные
4. точка обновляется

Выход:

1. обновлённый JSON точки

```
DELETE /routes/{route_id}/points/{point_id}
```

Вход:

1. идентификатор маршрута
2. идентификатор точки
3. JWT владельца маршрута или `admin`

Что происходит:

1. проверяется, что редактирование маршрута не запрещено бизнес-правилами
2. точка удаляется
3. значения `point_order` для оставшихся точек пересчитываются так, чтобы порядок оставался непрерывным

Выход:

1. служебный результат об успешном удалении

6.4. Впечатления и модерация

6.4.1. Назначение

Впечатление является расширением маршрута. Оно всегда строится на основе маршрута и включает в себя данные, нужные для показа в витрине, покупки и прохождения. Одному маршруту может соответствовать несколько впечатлений.

6.4.2. Состав данных впечатления

Впечатление должно содержать следующие поля:

1. `id`
2. `route_id`
3. `author_id`
4. `name`
5. `description`
6. `price`
7. `status`
8. `rating`
9. `popularity_score`
10. `moderation_comment`
11. `created_at`
12. `updated_at`

`rating` и `popularity_score` должны рассчитываться системой, а не передаваться пользователем как произвольные значения.

6.4.3. Правила расчёта `rating` и `popularity_score`

Поле `rating` должно рассчитываться автоматически на основе пользовательских отзывов.

Для MVP применяется следующее правило:

1. `rating` равен среднему арифметическому значений `review.rating` по всем отзывам данного впечатления
2. если у впечатления нет отзывов, значение `rating` принимается равным 0
3. пересчёт `rating` выполняется после создания нового отзыва

Поле `popularity_score` должно рассчитываться автоматически на основе аналитических событий.

Для MVP применяется следующее правило:

Сначала рассчитывается сырое значение популярности:

$\text{raw_popularity} = \text{views} * 1 + \text{starts} * 3 + \text{purchases} * 5 + \text{completions} * 4$

Затем поле `popularity_score` рассчитывается как нормализованное значение:

$\text{popularity_score} = \min(10, \text{raw_popularity} / 10)$

Где:

1. `views` — количество событий `impression_viewed`
2. `starts` — количество событий `impression_started`
3. `purchases` — количество событий `purchase_paid`
4. `completions` — количество событий `impression_completed`

Пересчёт `popularity_score` должен выполняться после поступления нового аналитического события, влияющего на популярность впечатления.

6.4.4. Пример JSON впечатления

```
{
  "id": 55,
  "route_id": 10,
  "author_id": 3,
  "name": "Историческая прогулка по Москве",
  "description": "Готовое впечатление на основе городского маршрута",
  "price": 499,
  "status": "published",
  "rating": 4.7,
  "popularity_score": 8.4,
  "moderation_comment": null,
  "created_at": "2026-04-08T10:00:00Z",
  "updated_at": "2026-04-08T10:00:00Z"
}
```

6.4.5. Статусы впечатления

Впечатление должно поддерживать следующие статусы:

1. `draft`
2. `on_moderation`
3. `published`
4. `rejected`
5. `archived`

Статус **draft** означает, что пользователь ещё не хочет публиковать впечатление. На этом этапе впечатление сохраняется как черновик, редактируется и может периодически пересохраняться клиентом, например по таймеру, чтобы не терялся прогресс заполнения формы.

6.4.5.1. Разрешённые переходы статуса

Для впечатления должны поддерживаться следующие переходы:

1. **draft**->**on_moderation** после **submit**, если владелец имеет роль **user**
2. **draft**->**published** после **submit**, если владелец имеет роль **author** или **admin**
3. **on_moderation**->**published** по решению **admin**
4. **on_moderation**->**rejected** по решению **admin**
5. **rejected**->**draft** после доработки владельцем
6. **rejected**->**on_moderation** или **rejected**->**published** при повторном **submit** без доработки, в зависимости от роли владельца
7. **published**->**archived** по решению владельца или **admin**

При редактировании впечатления в статусе **published** впечатление должно переводиться обратно в **draft**.

6.4.6. Правила валидации

1. **route_id** обязателен и должен ссылаться на существующий маршрут
2. **name** обязателен и должен содержать от 3 до 120 символов
3. **description** обязателен и должен содержать от 10 до 2000 символов
4. **price** обязателен и должен быть числом от 0 до 1000000
5. **status** должен принадлежать фиксированному набору значений
6. пользователь не должен иметь возможность вручную задавать **rating**
7. пользователь не должен иметь возможность вручную задавать **popularity_score**
8. **moderation_comment** может быть пустым, но при отклонении впечатления должен содержать от 1 до 1000 символов
9. маршрут, используемый для создания впечатления, должен либо принадлежать текущему пользователю, либо иметь статус **published**

6.4.7. Функциональные требования

1. система должна обеспечивать создание впечатления только на основе маршрута
2. система должна обеспечивать сохранение впечатления в статусе **draft**
3. система должна обеспечивать редактирование впечатления
4. система должна обеспечивать периодическое пересохранение черновика впечатления

5. система должна обеспечивать отправку впечатления на публикацию
6. система должна обеспечивать перевод впечатления пользователя в статус **on_moderation**
7. система должна обеспечивать публикацию впечатления автора без модерации
8. система должна обеспечивать публикацию впечатления администратором
9. система должна обеспечивать отклонение впечатления
10. система должна обеспечивать получение списка впечатлений
11. система должна обеспечивать получение карточки впечатления
12. система должна обеспечивать возврат отклонённого впечатления в **draft** после доработки
13. система должна обеспечивать перевод опубликованного впечатления в **draft** при его редактировании
14. система должна обеспечивать архивирование впечатления вместо физического удаления после публикации
15. система должна обеспечивать удаление впечатления

6.4.8. API-методы домена

POST /impressions

Вход:

1. `route_id`
2. `name`
3. `description`
4. `price`
5. JWT авторизованного пользователя

Что происходит:

1. проверяется существование маршрута
2. проверяется, что маршрут либо принадлежит текущему пользователю, либо имеет статус **published**
3. создаётся впечатление в статусе **draft**
4. впечатление сохраняется в базе

Выход:

1. JSON созданного впечатления

GET /impressions

Вход:

1. фильтры
2. параметры пагинации
3. параметр `scope`, принимающий значения `public`, `own`, `all`

Что происходит:

1. при `scope=public` возвращаются только `published`-впечатления
2. при `scope=own` пользователь получает только свои впечатления
3. при `scope=all` администратор получает впечатления всех статусов
4. данный API-метод используется для доменной работы с впечатлениями и отличается от витрины тем, что учитывает внутренние статусы, права доступа и служебные сценарии владельца и администратора

Выход:

1. массив впечатлений
2. служебные поля пагинации

```
GET /impressions/{impression_id}
```

Вход:

1. идентификатор впечатления
2. JWT обязателен для доступа к непубличному впечатлению

Что происходит:

1. проверяется доступ к объекту
2. система возвращает детальную карточку впечатления

Выход:

1. JSON впечатления
2. данные связанного маршрута
3. список отзывов
4. информация о покупке текущего пользователя, если пользователь авторизован

```
PATCH /impressions/{impression_id}
```

Вход:

1. изменяемые поля
2. JWT владельца впечатления или `admin`

Что происходит:

1. проверяется право изменения
2. валидируются поля
3. если впечатление находится в статусе **published**, оно переводится в **draft**
4. если впечатление находится в статусе **rejected**, оно остаётся в **draft** после сохранения изменений
5. впечатление обновляется
6. если впечатление находится в статусе **draft**, API-метод может использоваться клиентом для периодического автосохранения

Выход:

1. обновлённый JSON впечатления

```
POST /impressions/{impression_id}/submit
```

Вход:

1. идентификатор впечатления
2. JWT владельца впечатления

Что происходит:

1. проверяется, что впечатление принадлежит текущему пользователю
2. проверяется, что впечатление находится в статусе **draft** или **rejected**
3. проверяется, что маршрут впечатления находится в статусе **published**
4. если владелец впечатления имеет роль **user**, статус меняется на **on_moderation**
5. если владелец впечатления имеет роль **author** или **admin**, статус меняется на **published**

Выход:

1. обновлённый JSON впечатления

```
DELETE /impressions/{impression_id}
```

Вход:

1. идентификатор впечатления
2. JWT владельца впечатления или **admin**

Что происходит:

1. проверяется право на удаление
2. если впечатление находится в статусе **draft**, оно может быть удалено физически
3. во всех остальных случаях впечатление переводится в статус **archived**

Выход:

1. служебный результат об успешном удалении

```
GET /moderation/impressions
```

Вход:

1. JWT администратора

Что происходит:

1. система возвращает впечатления, ожидающие модерации

Выход:

1. массив впечатлений в статусе **on_moderation**

```
POST /moderation/impressions/{impression_id}/publish
```

Вход:

1. идентификатор впечатления
2. JWT администратора

Что происходит:

1. проверяется роль **admin**
2. проверяется, что впечатление находится в статусе **on_moderation**
3. проверяется, что маршрут впечатления находится в статусе **published**
4. статус меняется на **published**
5. формируется событие аналитики публикации

Выход:

1. обновлённый JSON впечатления

```
POST /moderation/impressions/{impression_id}/reject
```

Вход:

1. идентификатор впечатления

2. причина отклонения
3. JWT администратора

Что происходит:

1. проверяется роль `admin`
2. проверяется, что впечатление находится в статусе `on_moderation`
3. статус меняется на `rejected`
4. причина отклонения сохраняется в `moderation_comment`

Выход:

1. обновлённый JSON впечатления

6.5. Витрина впечатлений

6.5.1. Назначение

Витрина представляет собой публичный каталог впечатлений в статусе `published`. Витрина предназначена для клиентского экрана и отличается от доменных API-методов `/impressions` тем, что не возвращает черновики, объекты на модерации и иные служебные статусы.

6.5.2. Функциональные требования

1. система должна предоставлять список опубликованных впечатлений
2. система должна предоставлять карточку опубликованного впечатления
3. система должна поддерживать фильтрацию витрины
4. система должна поддерживать сортировку витрины
5. система должна скрывать непубличные статусы
6. система должна предоставлять список городов, в которых есть опубликованные впечатления

6.5.3. API-методы домена

```
GET /showcase/impressions
```

Вход:

1. параметры пагинации
2. фильтры по городу, цене, длительности
3. параметры сортировки

Что происходит:

1. выбираются только опубликованные впечатления

2. применяются фильтры и сортировка
3. возвращается краткое представление карточек

Выход:

1. массив карточек впечатлений
2. метаданные пагинации

```
GET /showcase/cities
```

Вход:

1. параметры не требуются

Что происходит:

1. система выбирает маршруты, связанные с опубликованными впечатлениями
2. система собирает уникальные города из найденных маршрутов
3. список городов сортируется по названию

Выход:

1. массив городов с полями `external_city_id` и `name`

```
GET /showcase/impressions/{impression_id}
```

Вход:

1. идентификатор впечатления

Что происходит:

1. проверяется, что впечатление опубликовано
2. возвращается карточка впечатления

Выход:

1. JSON карточки впечатления

6.6. Покупки

6.6.1. Состав данных покупки

Покупка должна содержать следующие поля:

1. `id`
2. `user_id`

3. `impression_id`
4. `impression_name`
5. `status`
6. `price_at_purchase`
7. `created_at`
8. `updated_at`

6.6.2. Пример JSON покупки

```
{
  "id": 500,
  "user_id": 12,
  "impression_id": 55,
  "impression_name": "Историческая прогулка по Москве",
  "status": "paid",
  "price_at_purchase": 499,
  "created_at": "2026-04-08T10:00:00Z",
  "updated_at": "2026-04-08T10:01:00Z"
}
```

6.6.3. Статусы покупки

1. `created`
2. `paid`
3. `failed`

6.6.4. Правила валидации

1. покупка должна ссылаться на существующего пользователя
2. покупка должна ссылаться на существующее впечатление
3. `status` должен принадлежать фиксированному набору значений
4. покупка непубличного впечатления должна быть запрещена
5. один пользователь может купить одно и то же впечатление только один раз
6. успешная покупка не имеет срока действия и сохраняет доступ к впечатлению бессрочно

6.6.5. Функциональные требования

1. система должна обеспечивать создание покупки
2. система должна обеспечивать запуск рау-заглушки

3. система должна обеспечивать изменение статуса покупки
4. система должна обеспечивать получение списка покупок пользователя

6.6.6. API-методы домена

POST /purchases

Вход:

1. `impression_id`
2. JWT пользователя

Что происходит:

1. проверяется доступность впечатления
2. проверяется, что у пользователя ещё нет покупки этого впечатления в любом статусе
3. создаётся покупка в статусе `created`
4. фиксируется цена на момент покупки

Выход:

1. JSON созданной покупки

POST /purchases/{purchase_id}/pay

Вход:

1. идентификатор покупки
2. JWT пользователя

Что происходит:

1. запускается учебная pay-заглушка
2. при успехе статус меняется на `paid`
3. после успешной оплаты пользователь получает бессрочный доступ к впечатлению
4. при неуспехе статус меняется на `failed`
5. покупка в статусе `failed` может быть оплачена повторно этим же методом
6. повторный вызов для уже оплаченной покупки возвращает покупку без изменений
7. формируется событие аналитики покупки

Выход:

1. обновлённый JSON покупки

GET /purchases/my

Вход:

1. JWT пользователя

Что происходит:

1. система возвращает покупки текущего пользователя

Выход:

1. массив покупок пользователя с названием связанного впечатления

6.7. Отзывы

6.7.1. Состав данных отзыва

Отзыв должен содержать следующие поля:

1. id
2. user_id
3. username
4. impression_id
5. rating
6. comment
7. created_at

6.7.2. Пример JSON отзыва

```
{
  "id": 700,
  "user_id": 12,
  "username": "denis_travel",
  "impression_id": 55,
  "rating": 5,
  "comment": "Очень понравился маршрут и подача материала",
  "created_at": "2026-04-08T12:00:00Z"
}
```

6.7.3. Правила валидации

1. **rating** должен быть целым числом от 1 до 5
2. **comment** должен содержать от 1 до 1000 символов
3. отзыв должен быть связан с существующим пользователем и впечатлением
4. один пользователь может оставить не более одного отзыва на одно впечатление
5. отзыв может быть оставлен только на впечатление в статусе **published**
6. пользователь может оставить отзыв только после завершения прохождения впечатления

6.7.4. Функциональные требования

1. система должна обеспечивать создание отзыва
2. система должна обеспечивать получение отзывов по впечатлению
3. система должна обеспечивать влияние отзывов на агрегированный рейтинг впечатления

6.7.5. API-методы домена

POST /impressions/{impression_id}/reviews

Вход:

1. **rating**
2. **comment**
3. JWT пользователя

Что происходит:

1. проверяется право оставить отзыв
2. проверяется, что впечатление опубликовано и пользователь завершил его прохождение

3. валидируются оценка и комментарий
4. отзыв сохраняется
5. рейтинг впечатления пересчитывается

Выход:

1. JSON созданного отзыва с именем пользователя

```
GET /impressions/{impression_id}/reviews
```

Вход:

1. идентификатор впечатления
2. параметры пагинации

Что происходит:

1. система возвращает отзывы по впечатлению

Выход:

1. массив отзывов с именами пользователей

6.8. Рекомендации

6.8.1. Назначение

Система должна формировать пользователю список впечатлений, которые наиболее подходят по простым эвристическим правилам

6.8.2. Правила подбора рекомендаций

Подбор рекомендаций должен выполняться в следующем порядке:

1. система определяет город, относительно которого должны строиться рекомендации
2. если пользователь явно выбрал город, используется переданный `city_external_id`
3. если город явно не выбран, текущий город пользователя определяется по геоданным, которые клиент передаёт в backend, как ближайший город из поддерживаемого справочника
4. если не переданы ни `city_external_id`, ни координаты, запрос отклоняется с ошибкой валидации
5. система формирует базовый набор кандидатов только из впечатлений в статусе `published`
6. из набора кандидатов исключаются впечатления, не подходящие по обязательным фильтрам пользователя
7. для оставшихся впечатлений рассчитывается `score`

8. впечатления сортируются по убыванию `score`
9. при равном `score` выше ставится впечатление с более высоким `rating`
10. при равном `score` и равном `rating` выше ставится впечатление с большей `popularity_score`
11. итоговый список ограничивается параметрами пагинации или лимитом выдачи

Обязательные фильтры подбора:

1. город пользователя или явно выбранный `city_external_id`
2. диапазон стоимости
3. диапазон длительности

6.8.3. Правила расчёта `score`

В MVP `score` рассчитывается на основе следующих факторов:

1. рейтинг впечатления
2. популярность впечатления

Поля `rating` и `popularity_score`, используемые в рекомендациях, должны предварительно рассчитываться системой по правилам, указанным в разделе 6.4.3.

Score является внутренним техническим показателем релевантности и не является пользовательской оценкой.

Для MVP применяется следующая формула:

`score=rating_score+popularity_score`

Где:

1. `rating_score` принимает значение от 0 до 10 и рассчитывается как `rating*2`
2. `popularity_score` принимает значение от 0 до 10 и рассчитывается по правилам, указанным в разделе 6.4.3

Город, диапазон стоимости и диапазон длительности используются как жёсткие фильтры и не участвуют в дополнительном начислении `score`.

6.8.4. Правила валидации

1. параметры фильтрации должны быть типизированы
2. минимальные и максимальные значения диапазонов должны быть согласованы
3. сортировка должна быть ограничена допустимыми значениями
4. в рекомендации должны попадать только опубликованные впечатления

6.8.5. Функциональные требования

1. система должна предоставлять рекомендации пользователю

2. система должна поддерживать фильтрацию
3. система должна поддерживать сортировку по score

6.8.6. API-методы домена

GET /recommendations

Вход:

1. city_external_id
2. latitude
3. longitude
4. min_price
5. max_price
6. min_duration
7. max_duration
8. параметры сортировки
9. JWT пользователя

Что происходит:

1. если передан city_external_id, система использует его
2. иначе текущий город пользователя определяется по переданным координатам как ближайший город из поддерживаемого справочника
3. если не переданы ни город, ни координаты, возвращается ошибка валидации
4. система выбирает опубликованные впечатления
5. применяет обязательные фильтры по городу, стоимости и длительности
6. рассчитывает score по правилам раздела 6.8.3
7. сортирует результат по убыванию score
8. при ничье учитывает rating, затем popularity_score

Выход:

1. массив рекомендованных впечатлений

6.9. Прогресс и realtime

6.9.1. Состав данных прогресса

Прогресс должен содержать следующие поля:

1. id

2. `user_id`
3. `impression_id`
4. `current_point`
5. `is_completed`
6. `updated_at`

6.9.2. Пример JSON прогресса

```
{
  "id": 900,
  "user_id": 12,
  "impression_id": 55,
  "current_point": 2,
  "is_completed": false,
  "updated_at": "2026-04-08T13:00:00Z"
}
```

6.9.3. Правила валидации

1. пользователь должен иметь право проходить впечатление
2. координаты должны быть валидными географическими координатами
3. сообщение WebSocket должно содержать `latitude` и `longitude`
4. `current_point` должен соответствовать структуре маршрута
5. пользователь не должен получать доступ к чужому прогрессу
6. пользователь может проходить впечатление только если впечатление имеет статус `published`
7. пользователь может проходить платное впечатление только при наличии покупки со статусом `paid`
8. бесплатное впечатление может проходиться без покупки
9. автор может проходить собственное впечатление без покупки
10. `admin` может получать доступ к прохождению любого впечатления для проверки
11. после завершения прохождения пользователь может начать прохождение впечатления повторно

6.9.4. Функциональные требования

1. система должна принимать координаты пользователя в реальном времени
2. система должна определять достижение точки маршрута
3. точка должна считаться достигнутой, если расстояние до неё меньше или равно 30 метрам

4. система должна обновлять прогресс прохождения
5. система должна сохранять прогресс в базе данных
6. система должна возвращать пользователю актуальное состояние прохождения
7. система должна рассчитывать расстояние между пользователем и точкой маршрута по формуле гаверсина
8. система должна поддерживать повторное прохождение впечатления после завершения предыдущего прохождения

6.9.5. API-методы домена

GET /progress/{impression_id}

Вход:

1. идентификатор впечатления
2. JWT пользователя

Что происходит:

1. система проверяет право пользователя на прохождение впечатления
2. система возвращает текущее состояние прохождения для пользователя

Выход:

1. JSON прогресса

WS /ws/{impression_id}

Вход:

1. JWT пользователя
2. сообщения вида:

```
{
  "latitude": 55.7539,
  "longitude": 37.6208
}
```

Что происходит:

1. система проверяет право пользователя на прохождение впечатления
2. если предыдущее прохождение уже завершено, то новое прохождение начинается при первом валидном сообщении координат и прогресс сбрасывается к начальному состоянию

3. система связывает соединение с пользователем и впечатлением
4. система определяет следующую ожидаемую точку маршрута
5. система вычисляет расстояние до точки по формуле гаверсинуса
6. при расстоянии не более 30 метров точка считается достигнутой
7. при достижении точки система обновляет `current_point`
8. при достижении последней точки система устанавливает `is_completed=true`

Выход:

1. обновлённый JSON прогресса
2. служебный признак достижения точки

При ошибке WebSocket-взаимодействия backend должен возвращать структурированное сообщение об ошибке в JSON-формате.

Пример ошибки WebSocket:

```
{
  "status": "error",
  "error": {
    "code": 403,
    "message": "Пользователь не может начать это впечатление"
  }
}
```

Для WebSocket-сценариев должны поддерживаться как минимум следующие типы ошибок:

1. 403Forbidden при ошибке авторизации или отсутствии права на прохождение
2. 422UnprocessableEntity при ошибке валидации входных координат
3. 404NotFound при обращении к несуществующему впечатлению

6.10. Аналитика

6.10.1. Назначение

Аналитика нужна для понимания того, как используется система, какие впечатления просматривают, покупают и проходят пользователи.

6.10.2. Состав события аналитики

Событие аналитики должно содержать:

1. `id`
2. `user_id`

3. event_type
4. entity_type
5. entity_id
6. metadata
7. timestamp

6.10.3. Пример JSON события

```
{
  "id": 1200,
  "user_id": 12,
  "event_type": "impression_started",
  "entity_type": "impression",
  "entity_id": 55,
  "metadata": {
    "source": "mobile_client"
  },
  "timestamp": "2026-04-08T13:10:00Z"
}
```

6.10.4. События аналитики

Система должна фиксировать как минимум:

1. impression_viewed
2. impression_started
3. impression_completed
4. purchase_created
5. purchase_paid
6. route_created
7. impression_published

6.10.5. Правила фиксации событий

События аналитики должны фиксироваться по следующим правилам:

1. **impression_viewed**: создаётся после каждого успешного получения карточки опубликованного впечатления
2. **impression_started**: создаётся при первом начале прохождения впечатления в рамках текущего незавершённого прохождения

3. `impression_completed`: создаётся при достижении последней точки маршрута
4. `purchase_created`: создаётся после успешного создания записи о покупке
5. `purchase_paid`: создаётся после успешного перевода покупки в статус `paid` и не должен создаваться повторно для одной и той же покупки
6. `route_created`: создаётся после успешного сохранения нового маршрута
7. `impression_published`: создаётся после успешной публикации впечатления администратором

6.10.6. Обязательные поля события

Для каждого события должны быть заполнены:

1. `event_type`
2. `timestamp`
3. `entity_type`
4. `entity_id`

Если действие связано с авторизованным пользователем, дополнительно должен быть заполнен `user_id`

Поле `entity_type` должно принимать одно из следующих значений:

1. `user`
2. `route`
3. `impression`
4. `purchase`
5. `progress`

Поле `metadata` должно использоваться для дополнительных параметров события, например:

1. источник события
2. способ вызова
3. дополнительные служебные данные для последующего анализа

6.10.7. Правила валидации

1. `event_type` должен принадлежать фиксированному набору значений
2. `metadata` должна быть сериализуемой структурой
3. событие должно иметь корректную временную метку

6.10.8. Функциональные требования

1. система должна автоматически создавать события аналитики при значимых действиях

2. система должна хранить события в базе данных
3. система должна предоставлять административный доступ к аналитическим данным
4. пользовательские данные не должны утекать через аналитику

6.10.9. API-методы домена

GET /analytics/events

Вход:

1. фильтры по типу события
2. фильтры по периоду
3. JWT администратора

Что происходит:

1. система выбирает события аналитики по условиям фильтрации

Выход:

1. массив событий аналитики

GET /analytics/summary

Вход:

1. период
2. JWT администратора

Что происходит:

1. система агрегирует события
2. возвращает сводные показатели

Выход:

1. агрегированная аналитика по выбранному периоду

6.11. BFF

6.11.1. Назначение

BFF API-методы нужны для того, чтобы frontend не собирал экран из большого числа отдельных вызовов

6.11.2. Функциональные требования

1. система должна предоставлять агрегированную карточку впечатления
2. система должна предоставлять агрегированную витрину

Экран текущего прохождения не требует отдельного BFF-метода: он собирается клиентом из агрегированной карточки впечатления и API прогресса.

6.11.3. API-методы домена

```
GET /bff/impressions/{impression_id}
```

Вход:

1. идентификатор впечатления
2. JWT пользователя, если нужно вернуть информацию о покупке текущего пользователя

Что происходит:

1. система собирает данные впечатления
2. добавляет маршрут
3. добавляет отзывы
4. добавляет информацию о покупке текущего пользователя

Выход:

1. агрегированная карточка впечатления

```
GET /bff/home
```

Вход:

1. параметры витрины
2. JWT пользователя, если экран персонализирован

Что происходит:

1. система собирает блоки для главного экрана
2. добавляет витрину и рекомендации

Выход:

1. агрегированная структура главного экрана

6.12. Интеграции

6.12.1. Назначение

Система должна поддерживать обращения к внешним API, например картографическим, и возвращать клиенту нормализованный ответ

6.12.2. Функциональные требования

1. backend должен выступать прокси между клиентом и внешним API
2. внешний ответ должен нормализоваться
3. внутренний формат ответа должен быть единообразным
4. в рамках MVP система использует одного выбранного провайдера геоданных

6.12.3. API-методы домена

GET /places/search

Вход:

1. поисковая строка
2. опционально город
3. опционально координаты пользователя

Что происходит:

1. backend отправляет запрос во внешний сервис
2. backend преобразует ответ в единый внутренний формат

Выход:

1. массив найденных мест в нормализованном виде

7. Общие требования к API

7.1. Формат ответов

Успешный ответ:

```
{
  "status": "success",
  "data": {},
  "error": null
}
```

Ответ при ошибке:

```
{
  "status": "error",
  "data": null,
  "error": {
```

```
    "code": 404,  
    "message": "Маршрут не найден"  
  }  
}
```

Ошибка валидации:

```
{  
  "status": "error",  
  "data": null,  
  "error": {  
    "code": 422,  
    "message": "Ошибка валидации",  
    "details": [  
      {  
        "field": "email",  
        "message": "Некорректный формат email"  
      }  
    ]  
  }  
}
```

Бизнес-конфликт состояния сущности:

```
{  
  "status": "error",  
  "data": null,  
  "error": {  
    "code": 409,  
    "message": "Маршрут нельзя изменить, потому что у него есть  
опубликованные впечатления"  
  }  
}
```

Все текстовые сообщения об ошибках возвращаются на русском языке. Это относится и к ошибкам валидации, которые формируются внешними библиотеками: например, при некорректном формате email сообщение приводится к виду «Некорректный формат email», а не возвращается в исходном англоязычном виде используемой библиотеки проверки.

7.1.1. Основные коды ошибок

Для основных типов отказов должны использоваться следующие коды:

1. **401Unauthorized**: отсутствует токен или токен не прошёл проверку
2. **403Forbidden**: у пользователя недостаточно прав для выполнения операции
3. **404NotFound**: запрошенная сущность не существует
4. **409Conflict**: операция невозможна из-за текущего состояния сущности
5. **422UnprocessableEntity**: входные данные не прошли валидацию

7.2. Требования к API

1. основной формат взаимодействия должен быть REST
2. для realtime-сценариев должен использоваться WebSocket
3. все ответы должны возвращаться в JSON
4. ошибки должны возвращаться в едином формате
5. текстовые сообщения об ошибках должны возвращаться на русском языке, включая ошибки валидации, формируемые внешними библиотеками (например, проверкой формата email)
6. API-методы должны проектироваться по доменам
7. входные данные должны валидироваться до выполнения бизнес-логики
8. заблокированный пользователь не должен получать доступ к защищённым API-методам
9. access token должен иметь ограниченный срок жизни
10. при проверке JWT должны проверяться подпись токена, срок действия токена и статус блокировки пользователя
11. при бизнес-конflikтах состояния сущности должен использоваться код ответа **409Conflict**

7.2.1. Общие правила пагинации и сортировки

Для всех ручек, возвращающих списки, должны использоваться единые правила:

1. пагинация должна задаваться параметрами `page` и `page_size`
2. `page` должен быть целым числом не меньше 1
3. `page_size` должен быть целым числом от 1 до 100
4. ответ списка должен содержать `items`, `page`, `page_size`, `total`
5. сортировка должна выполняться только по явно разрешённым полям конкретного API-метода
6. для `/routes` допустимы поля сортировки: `created_at`, `updated_at`, `name`
7. для `/impressions` допустимы поля сортировки: `created_at`, `updated_at`, `status`
8. для `/showcase/impressions` допустимы поля сортировки: `price`, `rating`, `popularity_score`, `created_at`
9. для `/recommendations` допустимы поля сортировки: `score`, `rating`, `popularity_score`

7.3. Список требуемых API

Метод	Путь	Домен	Доступ	Назначение
GET	/health	Сервис	Публичный	Проверка доступности сервиса
POST	/auth/register	Пользователи	Публичный	Регистрация пользователя
POST	/auth/login	Пользователи	Публичный	Авторизация пользователя
GET	/users/me	Пользователи	Авторизованный пользователь	Получение профиля текущего пользователя
PATCH	/admin/users/{user_id}/block	Пользователи	admin	Блокировка или разблокировка пользователя
PATCH	/admin/users/{user_id}/verify-author	Пользователи	admin	Перевод пользователя в роль author
POST	/routes	Маршруты	Авторизованный пользователь	Создание маршрута
POST	/routes/{route_id}/submit	Маршруты	Владелец маршрута	Отправка маршрута на публикацию
GET	/routes	Маршруты	Публичный или авторизованный пользователь	Получение списка маршрутов
GET	/routes/{route_id}	Маршруты	Публичный или авторизованный пользователь	Получение маршрута по идентификатору
PATCH	/routes/{route_id}	Маршруты	Владелец, admin	Редактирование маршрута
DELETE	/routes/{route_id}	Маршруты	Владелец, admin	Удаление маршрута
GET	/moderation/routes	Модерация	admin	Получение маршрутов на модерации
POST	/moderation/routes/{route_id}/publish	Модерация	admin	Публикация маршрута
POST	/moderation/routes/{route_id}/reject	Модерация	admin	Отклонение маршрута
POST	/routes/{route_id}/points	Точки маршрута	Владелец, admin	Добавление точки в маршрут
GET	/routes/{route_id}/points	Точки маршрута	Публичный или авторизованный пользователь	Получение точек маршрута
PATCH	/routes/{route_id}/points/{point_id}	Точки маршрута	Владелец, admin	Редактирование точки маршрута
DELETE	/routes/{route_id}/points/{point_id}	Точки маршрута	Владелец, admin	Удаление точки маршрута
POST	/impressions	Впечатления	Авторизованный пользователь	Создание впечатления
GET	/impressions	Впечатления	По правилам scope	Получение списка впечатлений
GET	/impressions/{impression_id}	Впечатления	По статусу и правам доступа	Получение карточки впечатления
PATCH	/impressions/{impression_id}	Впечатления	Владелец, admin	Редактирование впечатления
POST	/impressions/{impression_id}/submit	Впечатления	Владелец впечатления	Отправка впечатления на публикацию
DELETE	/impressions/{impression_id}	Впечатления	Владелец, admin	Удаление или архивирование впечатления

GET	/moderation/impressions	Модерация	admin	Получение впечатлений на модерации
POST	/moderation/impressions/{impression_id}/publish	Модерация	admin	Публикация впечатления
POST	/moderation/impressions/{impression_id}/reject	Модерация	admin	Отклонение впечатления
GET	/showcase/impressions	Витрина	Публичный	Получение витрины опубликованных впечатлений
GET	/showcase/cities	Витрина	Публичный	Получение списка городов с опубликованными впечатлениями
GET	/showcase/impressions/{impression_id}	Витрина	Публичный	Получение карточки опубликованного впечатления
POST	/purchases	Покупки	Авторизованный пользователь	Создание покупки, если впечатление ещё не куплено
POST	/purchases/{purchase_id}/pay	Покупки	Владелец покупки	Оплата через заглушку
GET	/purchases/my	Покупки	Авторизованный пользователь	Получение покупок текущего пользователя
POST	/impressions/{impression_id}/reviews	Отзывы	Авторизованный пользователь	Создание отзыва
GET	/impressions/{impression_id}/reviews	Отзывы	Публичный	Получение отзывов по впечатлению
GET	/recommendations	Рекомендации	Авторизованный пользователь	Получение рекомендаций
GET	/progress/{impression_id}	Прогресс	Авторизованный пользователь	Получение текущего прогресса
WS	/ws/{impression_id}	Realtime	Авторизованный пользователь	Передача координат и обновление прогресса
GET	/analytics/events	Аналитика	admin	Получение списка событий аналитики
GET	/analytics/summary	Аналитика	admin	Получение агрегированной аналитики
GET	/bff/impressions/{impression_id}	BFF	По правилам доступа к впечатлению	Получение агрегированной карточки впечатления
GET	/bff/home	BFF	Публичный или авторизованный пользователь	Получение агрегированных данных главного экрана
GET	/places/search	Интеграции	Публичный или авторизованный пользователь	Поиск мест через внешний API

8. Нефункциональные требования

8.1. Требования к отклику

Для локального учебного использования система должна удовлетворять следующим требованиям:

1. чтение одной сущности по идентификатору должно выполняться не более чем за 500 мс в 95% запросов

2. получение списка сущностей объёмом до 50 элементов должно выполняться не более чем за 1000 мс в 95% запросов
3. создание, изменение и удаление сущностей должно выполняться не более чем за 1500 мс в 95% запросов
4. ответ на WebSocket-сообщение с координатами должен возвращаться не более чем за 300 мс в 95% сообщений
5. указанные значения относятся к локальному запуску проекта без внешней боевой нагрузки

8.2. Общие требования к качеству

1. система должна поддерживать асинхронную обработку запросов
2. система должна корректно обрабатывать ошибки
3. система должна быть расширяемой
4. все входные данные должны валидироваться
5. непубличные данные не должны быть доступны без проверки прав

9. Требования к проверке качества

9.1. Общий подход

Проверка качества должна выполняться в соответствии с отдельным документом **Методика испытаний**. В методике испытаний должны быть описаны тест-кейсы, входные данные, шаги выполнения, ожидаемые результаты, а также отметка о том, какие проверки выполняются автоматически, а какие остаются ручными.

В рамках разработки допускается использовать подход test-first: сначала формулируется ожидаемое поведение, затем пишется тест, затем реализуется функциональность.

9.2. Минимальный состав проверок для методики испытаний

1. unit-тестирование сервисной логики
2. интеграционное тестирование API
3. интеграционное тестирование работы с базой данных
4. тестирование WebSocket-взаимодействия
5. smoke-тестирование ключевых пользовательских сценариев
6. ручное тестирование API

9.3. Сценарии, подлежащие отражению в методике испытаний

В методике испытаний должны быть отражены как минимум следующие сценарии:

1. успешная регистрация

2. ошибка регистрации при повторном email
3. успешная авторизация
4. отказ в доступе без токена
5. отказ в доступе при недостаточной роли
6. создание маршрута
7. ошибка создания маршрута при невалидных полях
8. добавление точки маршрута
9. ошибка при конфликте `point_order`
10. создание маршрута в статусе `draft`
11. отправка маршрута пользователя на публикацию с переводом в `on_moderation`
12. отправка маршрута автора на публикацию с переводом в `published`
13. создание впечатления в статусе `draft`
14. отправка впечатления пользователя на публикацию с переводом в `on_moderation`
15. отправка впечатления автора на публикацию с переводом в `published`
16. публикация маршрута администратором
17. публикация впечатления администратором
18. скрывание непубличных впечатлений из витрины
19. создание покупки
20. успешная оплата через заглушку
21. ошибка повторной покупки уже купленного впечатления
22. перевод опубликованного маршрута в `draft` при редактировании
23. перевод опубликованного впечатления в `draft` при редактировании
24. сохранение `moderation_comment` при отклонении маршрута и впечатления
25. отказ в доступе для заблокированного пользователя при валидном JWT
26. выдача рекомендаций
27. прохождение бесплатного впечатления без покупки
28. прохождение платного впечатления только при наличии покупки `paid`
29. запрет редактирования маршрута, если с ним связано опубликованное впечатление
30. запрет удаления или архивирования маршрута, если с ним связано опубликованное впечатление
31. создание отзыва только после завершения прохождения впечатления
32. возврат `409Conflict` при попытке редактировать маршрут, если с ним связано опубликованное впечатление

33. возврат `409Conflict` при попытке удалить маршрут, если с ним связано опубликованное впечатление
34. повторное прохождение впечатления после завершения предыдущего с корректным сбросом прогресса
35. обновление прогресса через `WebSocket`
36. фиксация аналитического события

9.4. Критерии успешности проверки качества

1. все критические бизнес-сценарии должны иметь автоматические тесты
2. все проверки прав доступа должны иметь негативные тест-кейсы
3. все проверки валидации должны иметь негативные тест-кейсы
4. `WebSocket`-сценарий должен быть проверен как минимум на один успешный и один ошибочный случай
5. формат API-ответов должен быть проверен тестами на успех и ошибку

4 Описание решения

4.1 Общий результат

В результате работы реализован прототип backend-сервиса `Travel backend`. Основной исполняемый модуль находится в `app/main.py`; доменные API размещены в каталоге `app/api`; модели базы данных описаны в `app/models.py`; Pydantic-схемы и валидация входных данных - в `app/schemas.py`; бизнес-правила и вспомогательные вычисления - в `app/services.py`. Автоматические проверки размещены в `tests/test_smoke.py`.

Исходный код проекта размещён в GitHub-репозитории: <https://github.com/NobodyForYouCreator/cursach>.

Фактические артефакты результата перечислены в Таблице 4.1. Они показывают, что проект представлен не только текстовым описанием, но и работающим кодом.

4.2 Выбор технологий

Стек проекта выбран с учётом требований к API, ограниченного времени разработки, знаний и пригодности к дальнейшему развитию. Используемые технологии приведены в Таблице 4.2.

Таблица 4.1: Артефакты разработанного решения

Артефакт	Назначение
<code>app/main.py</code>	Инициализация FastAPI-приложения, подключение роутеров, обработчиков ошибок и жизненного цикла.
<code>app/api/*</code>	Набор REST и WebSocket endpoint по доменам системы.
<code>app/models.py</code>	Реляционная модель данных: пользователи, маршруты, точки, впечатления, покупки, отзывы, прогресс и события аналитики.
<code>docker-compose.yml</code>	Локальный запуск PostgreSQL для разработки и проверки.
<code>tests/test_smoke.py</code>	Интеграционные проверки основных пользовательских, авторских и административных сценариев.

Таблица 4.2: Используемые технологии

Технология	Причина выбора
FastAPI	Позволяет быстро разрабатывать typed REST API, автоматически формирует OpenAPI-документацию и поддерживает асинхронные обработчики.
SQLAlchemy Async	Даёт ORM-слой для работы с реляционной моделью и поддерживает асинхронный доступ к данным.
PostgreSQL	Подходит для связанных доменных данных, транзакций, ограничений уникальности и дальнейшего промышленного развёртывания.
Pydantic	Используется для описания входных и выходных схем, нормализации строк и проверки ограничений.
JWT	Позволяет реализовать stateless-авторизацию для защищённых API-методов.
Pytest и HTTPX	Используются для автоматической проверки API-контракта и пользовательских сценариев.

4.3 Архитектура приложения

Приложение построено как модульный backend по доменам. Внешний слой представлен API-роутерами FastAPI. Каждый роутер отвечает за одну функциональную область: авторизацию, маршруты, впечатления, покупки, отзывы, рекомендации, прогресс, аналитику, BFF или интеграции. Такой подход уменьшает связанность кода и упрощает проверку соответствия реализации техническому заданию.

Внутри приложения выделены следующие слои:

- слой API принимает HTTP- или WebSocket-запросы, вызывает проверки доступа и возвращает нормализованные ответы

- слой схем описывает контракты входных и выходных данных
- слой моделей описывает таблицы базы данных и связи между сущностями
- сервисный слой содержит переиспользуемые бизнес-правила: проверку владельца, расчёт рейтинга, расчёт популярности, проверку городов и прогресс по координатам
- слой инфраструктуры отвечает за конфигурацию, подключение к базе данных, JWT и обработку ошибок

Для взаимодействия с frontend в приложении подключён CORS middleware. Он разрешает локальному клиенту обращаться к backend во время разработки и не меняет внутренние правила авторизации: защищённые API-методы по-прежнему требуют валидный JWT.

Такое разделение не является избыточным для проекта: оно позволяет держать пользовательские сценарии в роутерах, а общие правила не дублировать между доменами.

4.4 Модель данных

Модель данных отражает жизненный цикл продукта. Центральной сущностью является **Impression**: оно создаётся автором на основе **Route**, публикуется в витрине, покупается пользователями, получает отзывы, рейтинг, популярность и прогресс прохождения. Маршрут, в свою очередь, содержит упорядоченные **RoutePoint**, без которых его нельзя отправить на публикацию.

В базе данных реализованы следующие важные ограничения:

- уникальность имени пользователя и email
- уникальность порядка точки внутри одного маршрута
- запрет повторной покупки одного впечатления одним пользователем
- запрет повторного отзыва одного пользователя на одно впечатление
- уникальность прогресса пользователя по конкретному впечатлению

Эти ограничения вынесены на уровень модели данных, потому что они являются инвариантами предметной области и не должны зависеть только от логики отдельных API-методов.

4.5 Реализованные пользовательские сценарии

В текущем прототипе покрыт полный основной сценарий: пользователь регистрируется, создаёт маршрут, добавляет точку, отправляет маршрут на модерацию, администратор публикует маршрут, пользователь создаёт впечатление, публикует его через модерацию, покупает, оплачивает через заглушку, проходит по координатам и оставляет отзыв. После этого система пересчитывает рейтинг и фиксирует аналитические события.

Также реализован авторский сценарий: администратор переводит пользователя в роль **author**, после чего маршруты и впечатления автора публикуются сразу после отправки, если выполняются бизнес-условия. Это соответствует требованию о верифицированных авторах.

Административный сценарий включает блокировку пользователей, назначение роли автора, просмотр объектов на модерации, публикацию и отклонение маршрутов и впечатлений, а также просмотр аналитической сводки.

4.6 API-контракт и обработка ошибок

Все успешные ответы возвращаются в формате:

```
{
  "status": "success",
  "data": ...,
  "error": null
}
```

Ошибки возвращаются в симметричном формате:

```
{
  "status": "error",
  "data": null,
  "error": {
    "code": 422,
    "message": "Ошибка валидации",
    "details": [...]
  }
}
```

Единый формат ответа важен для будущего frontend: клиенту не нужно по-разному обрабатывать ошибки в разных доменах. Для прикладных конфликтов используется код 409, для отсутствующих объектов - 404, для ошибок прав доступа - 401 и 403, для ошибок входных данных - 422.

4.7 Realtime-прогресс

Для прохождения впечатления реализован WebSocket endpoint `/ws/{impression_id}`. Клиент передаёт текущие координаты пользователя, после чего backend сравнивает их с координатами следующей точки маршрута. Если пользователь находится достаточно близко к точке, прогресс обновляется, а в аналитику записывается событие старта или завершения впечатления.

Расстояние между координатами вычисляется по формуле гаверсина:

$$d = 2R \cdot \arctan 2 \left(\sqrt{a}, \sqrt{1 - a} \right), \quad (1)$$

где R - радиус Земли, а a вычисляется через разности широты и долготы двух точек. В

проекте эта формула используется как простое и достаточное приближение для городского сценария.

4.8 Рекомендации, витрина и аналитика

Рекомендации реализованы без машинного обучения, что соответствует границам проекта. Система выдаёт опубликованные впечатления по городу и сортирует их по доступным признакам, например рейтингу или популярности. Если клиент передаёт координаты, backend выбирает ближайший город из поддерживаемого справочника с помощью уже используемой функции расчёта расстояния. В справочник включены основные города MVP, включая Москву, Санкт-Петербург, Казань, Екатеринбург, Новосибирск и Калининград.

Для витрины дополнительно реализован API-метод `GET /showcase/cities`. Он возвращает только те города, для которых есть опубликованные впечатления. Это позволяет frontend строить фильтр витрины по фактическим данным, а не показывать пользователю пустые направления.

В ответах для клиентских экранов используются небольшие денормализованные поля. Список покупок пользователя возвращает название связанного впечатления, а отзывы возвращаются вместе с именем автора отзыва. Эти поля не создают новых сущностей в базе данных, но уменьшают количество дополнительных запросов со стороны frontend.

Аналитика хранится как набор событий с типом, сущностью, идентификатором сущности, пользователем и дополнительными метаданными. Такой формат расширяем: при появлении frontend можно добавлять новые события без изменения уже существующих таблиц маршрутов, покупок или отзывов.

4.9 Проверка качества

Качество реализации проверяется автоматическими интеграционными тестами. Они запускают приложение через `TestClient`, поднимают тестовую схему базы данных и проходят основные сценарии end-to-end. Проверяются не только успешные запросы, но и отрицательные случаи:

- повторная покупка одного впечатления возвращает 409
- заблокированный пользователь получает 403
- невалидные входные данные возвращают 422

- неизвестный endpoint возвращает 404 в едином формате
- маршрут без точек нельзя отправить на публикацию
- впечатление нельзя опубликовать, если его маршрут не опубликован
- BFF-метод возвращает детальную карточку впечатления вместе с маршрутом и точками
- витрина и рекомендации возвращают только опубликованные впечатления

Таким образом, тесты проверяют не отдельные функции, а соответствие реализации пользовательским сценариям и API-контракту.

Дополнительно работоспособность backend-сервиса проверялась вручную через автоматически сформированную Swagger-документацию. На Рисунках [4.1–4.3](#) приведены примеры ручной проверки API: доступность endpoint, успешное выполнение запроса регистрации и обработка ошибки валидации во входных данных.

Service status			^
GET	/health	Health	▼
Authentication			^
POST	/auth/register	Register	▼
POST	/auth/login	Login	▼
Users and administration			^
GET	/users/me	Me	▼
PATCH	/admin/users/{user_id}/block	Block User	▼
PATCH	/admin/users/{user_id}/verify-author	Verify Author	▼
Routes			^
POST	/routes	Create Route	▼
GET	/routes	List Routes	▼
GET	/routes/{route_id}	Get Route	▼
PATCH	/routes/{route_id}	Update Route	▼
DELETE	/routes/{route_id}	Delete Route	▼
POST	/routes/{route_id}/submit	Submit Route	▼
GET	/moderation/routes	Moderation Routes	▼
POST	/moderation/routes/{route_id}/publish	Publish Route	▼
POST	/moderation/routes/{route_id}/reject	Reject Route	▼
POST	/routes/{route_id}/points	Create Point	▼
GET	/routes/{route_id}/points	List Points	▼
PATCH	/routes/{route_id}/points/{point_id}	Update Point	▼
DELETE	/routes/{route_id}/points/{point_id}	Delete Point	▼

Рис. 4.1: Фрагмент Swagger-документации с группами endpoint и методами домена маршрутов

no parameters

Request body required application/json

Edit Value | Schema

```
{
  "username": "hsenerd",
  "email": "nerdehse.ru",
  "password": "ilovehse123!"
}
```

Execute Clear

Responses

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8001/auth/register' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "username": "hsenerd",
    "email": "nerdehse.ru",
    "password": "ilovehse123!"
  }'
```

Request URL

http://127.0.0.1:8001/auth/register

Server response

Code	Details
200	Response body <pre>{ "status": "success", "data": { "access token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIzIiwiaWF0IjoxNzgwNTgwMDc3LCJleHAiOiE3ODAxODcyNzd9.k3bHkn7PaBIVilagCx49rLHCxpgV_08wgPoNCfvjLAs", "token_type": "bearer", "user_id": 3, "username": "hsenerd", "email": "nerdehse.ru", "role": "user" }, "error": null }</pre> Response headers <pre>content-length: 292 content-type: application/json date: Thu, 04 Jun 2026 13:34:36 GMT server: uvicorn</pre>

Responses

Рис. 4.2: Успешная регистрация пользователя через метод POST /auth/register

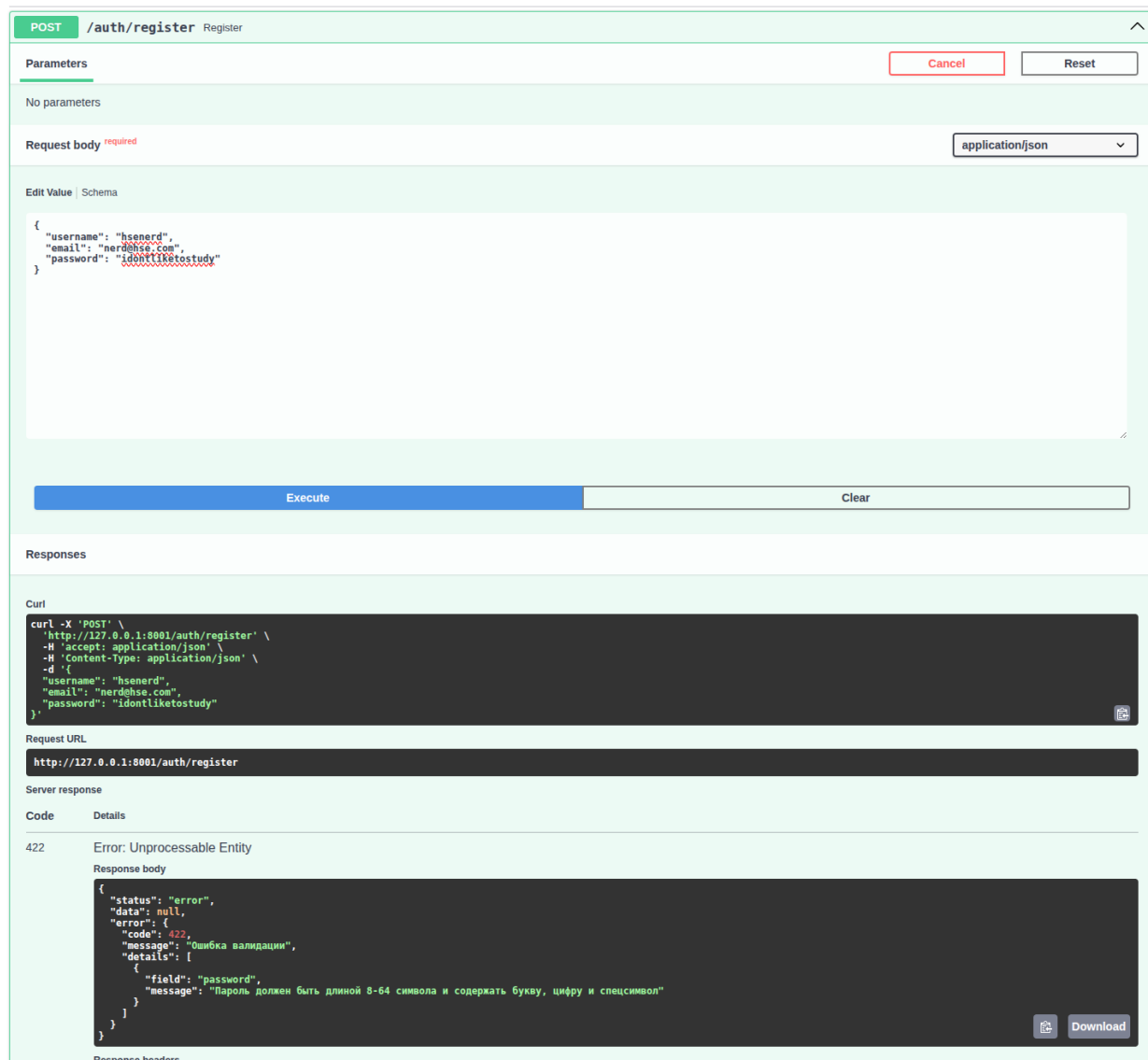


Рис. 4.3: Проверка ошибки валидации при некорректном пароле в методе POST `/auth/register`

5 Рефлексия проделанной работы

Изначальное техническое задание оказалось достаточно широким: оно включало не только CRUD-операции, но и роли, модерацию, витрину, покупки, realtime-прогресс, аналитику, BFF и интеграции. На практике самым важным решением стало разделение проекта по доменам. Без этого логика маршрутов, впечатлений, покупок и прогресса быстро смешалась бы в одном модуле.

При реализации пришлось уточнить границы MVP. Реальные платёжные системы, полноценные ML-рекомендации и сложная мультимедийная платформа были исключены из проекта, потому что они существенно увеличивали бы объём работы, но не являлись необходимыми для демонстрации основной backend-логики. Вместо этого была реализована рау-заглушка, rule-based рекомендации и прокси/локальная заглушка для поиска мест.

Сроки разработки в целом соответствовали выбранному MVP, но часть времени ушла не на написание endpoint, а на согласование инвариантов: какие статусы допустимы, кто может видеть черновики, когда маршрут можно редактировать, какие сценарии должны завершаться конфликтом. Это нормальная особенность backend-проектов: значительная часть сложности находится не в синтаксисе API, а в правилах предметной области.

6 Заключение

В ходе курсового проекта был разработан backend-сервис для трэвел-приложения с маршрутами и впечатлениями. Реализация покрывает основные требования технического задания: регистрацию и авторизацию пользователей, роли, маршруты, точки маршрута, впечатления, модерацию, витрину, покупки, отзывы, рекомендации, realtime-прогресс, аналитику, BFF-методы и интеграционный endpoint поиска мест. В реализации также учтены потребности клиентского приложения: включён CORS для локального frontend, добавлен список городов витрины, а ответы покупок и отзывов дополнены данными, которые нужны для отображения экранов без лишних запросов. Сервис имеет единый API-контракт, реляционную модель данных, проверку прав доступа и автоматические интеграционные тесты. Полученный прототип можно использовать как основу для дальнейшей разработки frontend, подключения реальных платёжных систем, внешних картографических API и более сложных рекомендательных механизмов.

7 Использование инструментов искусственного интеллекта

При подготовке отчёта инструменты искусственного интеллекта использовались только как вспомогательное средство для оформления документа: приведения текста к единому стилю, подготовки LaTeX-разметки, настройки списков, таблиц, блоков кода и структуры разделов. Содержательные решения по предметной области, техническому заданию, архитектуре backend-сервиса, реализованным функциям и выводам работы не формировались искусственным интеллектом.

Техническое задание, описание реализованного программного продукта и результаты проверки качества отражают фактическое содержание проекта. Обращение к искусственному интеллекту было строго ограничено рамками, заданными научным руководителем. Использование ИИ не заменяло самостоятельную разработку программного обеспечения, анализ требований и проверку соответствия реализации заявленным сценариям.

А Структура реализованных API-доменов

Таблица А.1: Соответствие API-модулей доменам системы

Модуль	Ответственность
<code>health.py</code>	Проверка доступности сервиса.
<code>auth.py</code>	Регистрация и вход пользователя.
<code>users.py</code>	Получение профиля, блокировка пользователя, назначение роли автора.
<code>routes.py</code>	CRUD маршрутов, отправка на публикацию, модерация, CRUD точек маршрута.
<code>impressions.py</code>	CRUD впечатлений, публикация, отклонение, архивирование, доменный просмотр.
<code>showcase.py</code>	Публичная витрина опубликованных впечатлений и список городов, в которых есть опубликованные впечатления.
<code>purchases.py</code>	Создание покупки, оплата через заглушку, список покупок пользователя с названием впечатления.
<code>reviews.py</code>	Создание и просмотр отзывов с именами пользователей.
<code>recommendations.py</code>	Rule-based рекомендации по явно выбранному или ближайшему по координатам городу.
<code>progress.py</code>	Получение прогресса и WebSocket-прохождение впечатления.
<code>analytics.py</code>	Список событий и агрегированная аналитика.
<code>bff.py</code>	Укрупнённые ответы для экранов frontend.
<code>places.py</code>	Поиск мест через внешний API или локальную заглушку.
<code>common.py</code>	Вспомогательные функции API-слоя: сериализация ответов и фиксация изменений.
<code>impression_details.py</code>	Сборка детальной карточки впечатления: маршрут, точки, отзывы с именами пользователей, покупка.